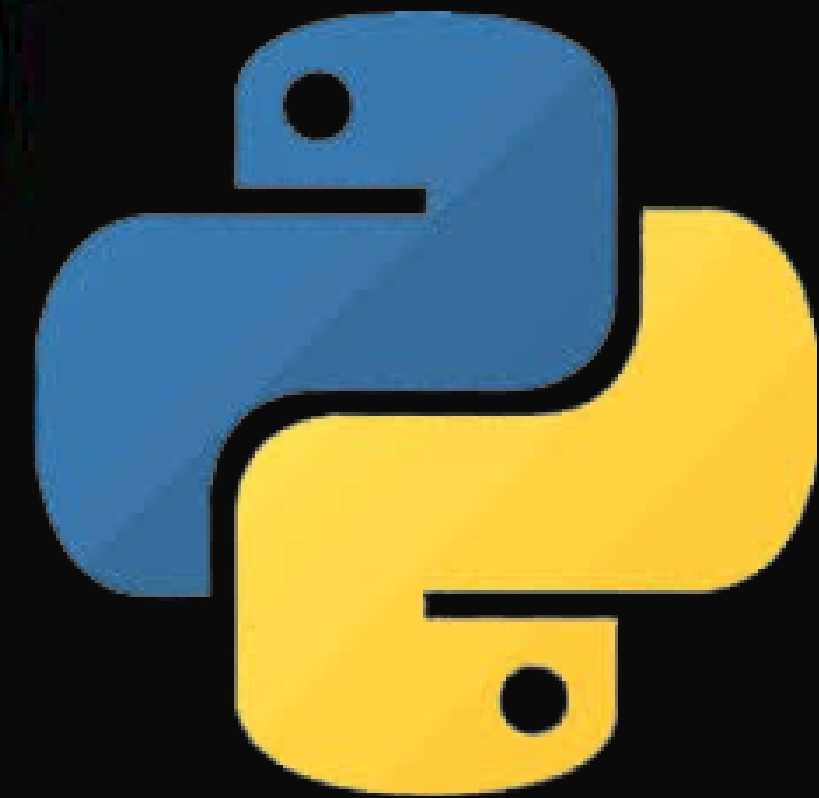
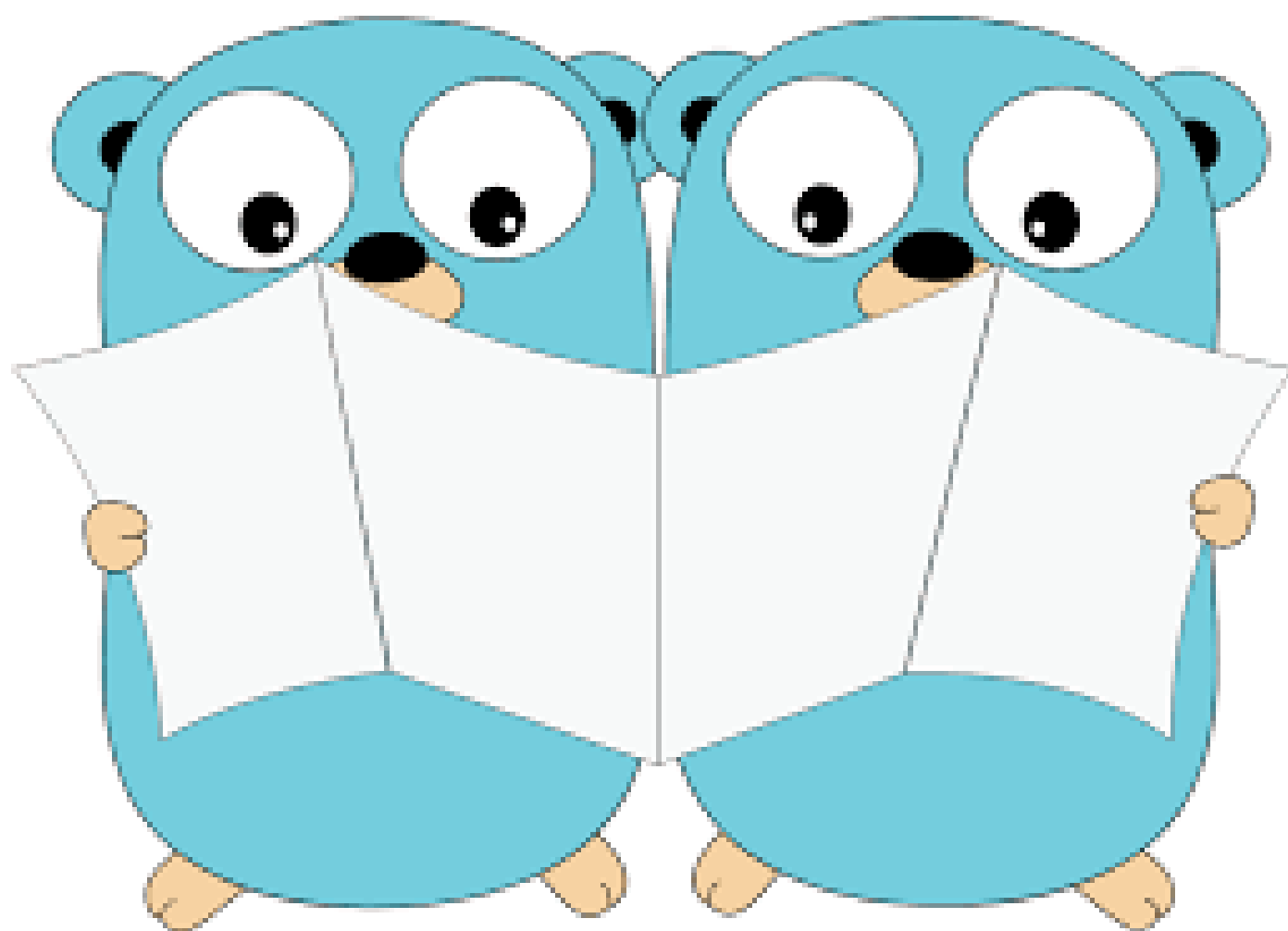


Workflow orchestration with Temporal



Contents



1 What is a workflow

2 What is temporal

3 Temporal architecture overview

4 How it works

5 Failure Case handling

6 Real World Use Cases

7 Project Github Link

What is a Workflow

A workflow is a multistage process involves a long running process that spans multiple services and may need human interaction in a distributed environment that requires transactional properties. such as user signup, order processing, video processing, multistage batch job processing etc.

Point to point communication is unreliable because of unpredictable nature of distributed systems . So developers require to write a lot of custom ad hoc retry backoff logic, persistent state storage adding more complexity to microservice and adding burden of managing extra code.

What is Temporal

Temporal is a durable workflow orchestration platform that help build and run long running reliable workflows (even for years long) and business logic in native language without worrying about failures and automatic retries and state persistence

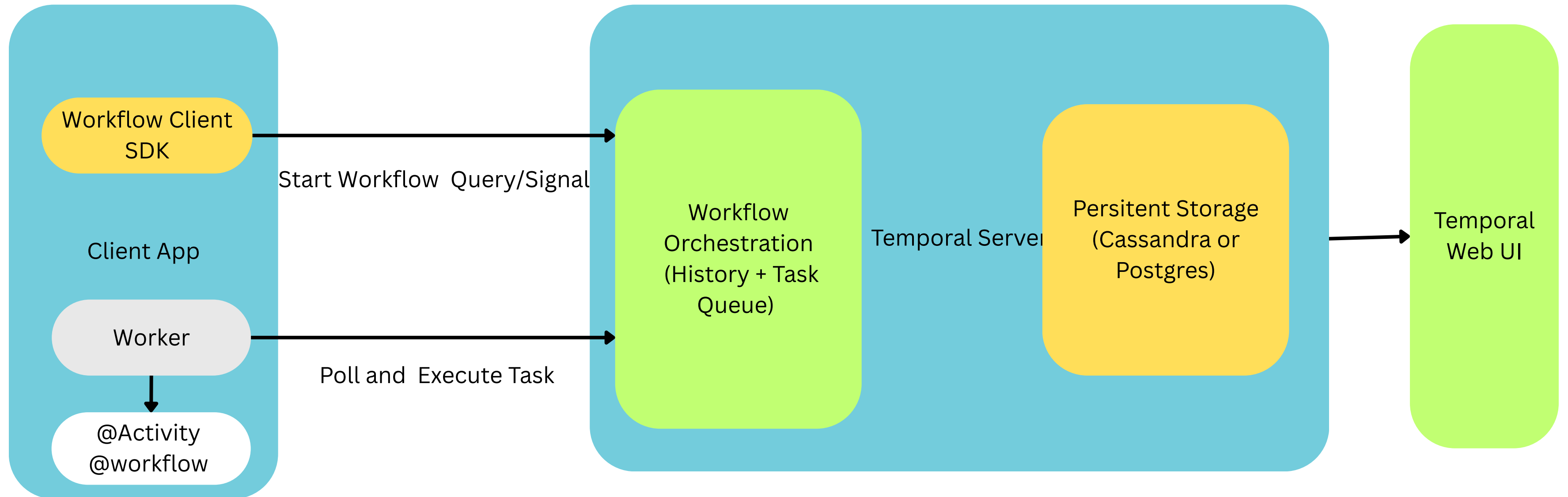
- It abstracts complexities by maintaining complete application state and retrying from last point of failure.
- No need of queues, pub/sub systems and schedulers.
- It manages task execution, event handling and timing on its own.
- If a failure occurs, the Workflow Execution can resume from the last recorded event, ensuring that progress isn't lost.

Temporal Architecture Overview

Components of temporal and roles :-

- **Workflows** :- application logic , defining sequence of steps the application will take. written in user programming language like go , java, python, js etc. Deterministic ensuring consistent outcomes with same inputs. Executed on your infrastructure not on temporal server.
- **Activities** :- smallest units of business logic written in application, designed to perform specific tasks that may fail, like sending email, updating tables in multiple dbs, connecting with other service. activities are run on your infra and managed by temporal server.
- **Workers** :- these are processes that execute the workflows and activities on your infra and they communicate constantly with temporal server to coordinate execution of tasks.
- **Temporal Server** :- Master server acts like a supervisor for your apps or services. It can be self hosted or as a SAAS. It maintains workflow execution history.

Temporal Architecture



How it works

- 1 **Client Starts Workflow** - client app sends a workflow request to temporal server with input data
- 2 **Server stores requests** - Temporal saves the workflow in database and creates a task
- 3 **Worker picks task** - worker polls temporal and gets workflow task
- 4 **Workflow code runs** - worker executes workflow code
- 5 **Workflow calls activities** - workflow schedules activities (business logic like db, api call)
- 6 **Activities execute** - workers pick up and run activity code, perform real business ops
- 7 **Results recorded** - each step and result is saved in database
- 8 **Workflow completes** - after all activities finish, workflow returns final result to client

Failure Case Handling

1. **Activity Fails** - If activity throws error, temporal automatically retries with exponential backoff
2. **Worker crashes** - if worker dices in mid of execution, another worker picks up from last saved state
3. **Server Restarts** - all workflow state is in DB, so sexcution resumes after server comes back.
4. **Retry limits reached** - after max retries, activity fails permantly and workflow can handle the error.
5. **Timeout handling** - if activity takes too long, temporal enforces timeouts and can retry gracefully.
6. **Manual intervention** - via, i, you can retry failed workflows , terminate stuck ones. or signal running workflows.

Real world Use Cases

1. Payment Processing
2. multi stage ,multi service Inventory Update
3. notification sending.
4. File synchronization.
5. multi stage batch job processing
6. extremely long running jobs like model training, data lake processing.

Project Github Link

Github link :-

<https://github.com/subpxl/engineering-deep-dive/distributed-systems/temporal-workflows>

[Github.com/subpxl](https://github.com/subpxl)

Shubham Panchal

